Technical Design: Youth Yellow Pages Platform

Verified talent and opportunity directory with an embedded MCP server and an in-house RAG engine that retrieves and scores people by skills. Version 1 draft.

1. Goal and scope

Build a searchable yellow pages for Nigeria, South Africa, Kenya and Ghana covering young talent, small businesses, employers and opportunities. Three ways in, one engine: a web and mobile app, a REST API, and an MCP server so AI agents can query the directory through typed tools. The engine retrieves candidates with RAG, scores them against a requirement, and explains each score using only stored evidence.

Success is measured by verified paid placements and retention, not by listings or sign-ups.

2. Architecture

Layer	Components	Notes
Clients	Next.js web, Flutter app, external AI agents via MCP; WhatsApp or USSD in a later phase	Low-bandwidth friendly
API	Django + DRF, auth, rate limits	REST under /v1
MCP server	Python MCP SDK (FastMCP), streamable HTTP at /mcp	Calls the same service layer as REST
Engine	Query parser, hybrid retriever, scorer, reranker, explainer	Single service layer, no logic in the doors
Data	Supabase Postgres with pgvector, Redis, object storage	Row-level security on profile data
Workers	Celery: skill extraction, embedding, verification, reindex	Redis as broker
Models	AWS Bedrock: an embedding model (for example Titan Text Embeddings v2) and an LLM for extraction and explanation	Swappable behind an interface

Design rule: one service layer, three doors (REST, MCP, admin). All permission checks live in the service layer, so MCP can never see more than REST.

3. Data model

Table	Key fields
users	id, role (youth, employer, business, admin), country, consent flags
profiles	user_id, headline, bio, location, availability, languages, embedding vector(1024)
skills	id, name, aliases, taxonomy path, embedding. Canonical taxonomy, about 300 skills at launch
profile_skills	profile_id, skill_id, level 1 to 5, years, last_used, source (self, assessment, evidence, endorsed)
evidence	profile_id, type (project, certificate, assessment, employer_review), url, verified, verified_by, date
listings	type (business, service, job, internship, apprenticeship), title, description, required skills with level and must-have flag, location, status, embedding
matches	profile_id, listing_id, score, breakdown json, outcome (viewed, applied, interview, hired, retained_6m)
audit_log	actor, tool or endpoint, args hash, timestamp

4. Ingestion pipeline

1. Profile arrives by form, CV upload, or portfolio and GitHub link.

2. LLM extracts skills as JSON against the fixed taxonomy. Unknown skills go to a review queue, never straight into the taxonomy.

3. Normalise: alias match first, then embedding similarity above a threshold (start at 0.80, tune on data).

4. Build a profile document from headline, skills, evidence summaries. Embed and store in pgvector.
5. Verify evidence: link reachable, repo ownership via OAuth, certificate lookup, employer confirmation.
6. Re-embed on change. Nightly reindex of stale vectors.

5. Retrieval (RAG) flow

1. Parse the query (free text or structured) into must-have skills, nice-to-have skills, location, availability and level.
2. Hard filters in SQL: country, active status, availability, candidate consent.
3. Candidate generation: top 200 by vector similarity (HNSW, cosine) plus top 200 by keyword (tsvector and trigram). Merge with reciprocal rank fusion.
4. Score the top 100 with the model in section 6.
5. Rerank the top 30 with an LLM or cross-encoder reading profile plus evidence against the requirement. Output structured JSON.
6. Return top k with matched skills, missing skills, evidence links and a suggested next project.

Grounding rule: explanations may cite only fields present in retrieved records. No invented skills, no inferred demographics.

6. Skill scoring model

```
final = 100 x (0.35 x S_sem + 0.30 x S_skill + 0.20 x S_evid + 0.10 x S_fresh + 0.05 x S_fit)
```

Term	Definition
S_sem	Cosine similarity of requirement and profile embeddings, min-max scaled across the candidate set
S_skill	Weighted coverage: sum of w x min(level / required_level, 1) over required skills, divided by sum of w. Must-have w = 1.0, nice-to-have w = 0.4
S_evid	Share of matched skills backed by evidence, weighted by source: assessment 1.0, employer review 0.9, verified project 0.8, certificate 0.5, self-declared 0.2
S_fresh	Exponential decay on last use of each skill, half-life 24 months, floor 0.3 so career gaps are not punished
S_fit	Location, availability, language, pay range fit

- Gate: if any must-have skill has zero coverage, cap the final score at 40.
- Bands: 80 and above strong, 60 to 79 good, 40 to 59 partial, below 40 weak. Return a confidence value based on evidence coverage.
- All weights are starting values. Calibrate against recruiter-labelled pairs before launch.

```
from math import prod

def skill_coverage(req, prof):
    num = den = 0.0
    for r in req.skills:
        w = 1.0 if r.must else 0.4
        lvl = prof.level(r.skill_id)
        num += prod((w, min(lvl / r.level, 1.0)))
        den += w
    return num / den if den else 0.0
```

Fairness controls

- Name, gender, age, photo and school prestige are excluded from all scoring features.
- Monthly report of score distribution and placement rate by gender, country and region.
- Scores are decision support. Employers see the breakdown, not only the number.

7. MCP server

Tool	Purpose	Access
search_talent	Rank consented candidates for a requirement. Returns score and breakdown	read
get_profile	Public profile and verified evidence, consented fields only	read
score_candidate	Score one profile against one requirement, with gaps	read
search_opportunities	Match a profile or query to listings	read
get_skill_gap	Missing skills for a listing and projects to close them	read
list_skills	Taxonomy lookup	read
create_listing	Publish a job, internship or service listing	write, employer only
record_outcome	Log applied, interview, hired, retained	write, scoped

- Resources (read-only): skills taxonomy, supported countries, scoring rubric.
- Prompts: shortlist_for_role, career_path_for_profile.
- Auth: OAuth 2.1 bearer tokens with scopes (read:public, read:candidates, write:listings). API keys for employers in phase 1.
- Safety: profile text is untrusted data. Strip instruction-like content, never place it in a system prompt, cap result sizes.
- Contact details are released only after the candidate accepts. Write tools need an explicit scope. Every call is logged and rate limited per token.

```
from mcp.server.fastmcp import FastMCP
from core.services import search, scoring

mcp = FastMCP("yellowpages")

@mcp.tool()
def search_talent(query: str, country: str | None = None,
                  skills: list[str] | None = None,
                  min_score: int = 40, limit: int = 10) -> list[dict]:
    """Rank consented candidates for a requirement."""
    reqs = search.parse_requirement(query, skills)
    pool = search.retrieve(reqs, country=country, k=100)
    ranked = scoring.rank(pool, reqs)
    return [r.public_dict() for r in ranked if r.score >= min_score][:limit]
```

8. REST endpoints

Endpoint	Purpose
POST /v1/profiles	Create or update a profile, triggers extraction and embedding
POST /v1/profiles/{id}/evidence	Attach evidence for verification
GET /v1/search/talent	Employer search, same engine as the MCP tool
GET /v1/search/opportunities	Youth search for listings
POST /v1/listings	Create a listing, employer verification required
POST /v1/matches/{id}/outcome	Record hire and retention outcomes
/mcp	MCP server endpoint

9. Privacy, security and trust

- Field-level consent. Candidates choose what is visible and can delete their data on request.
- Encryption in transit and at rest. Supabase row-level security on profiles and evidence.

- Collect the minimum. No national ID numbers in the scoring path.
- Check obligations under Kenya's Data Protection Act, Nigeria's Data Protection Act, South Africa's POPIA and Ghana's Data Protection Act with local counsel before launch.
- Employer verification (business registration check) and listing review to block fake jobs. Report button on every listing and profile.

10. Evaluation

- Gold set: about 200 candidate and requirement pairs rated 0 to 3 by recruiters.
- Offline metrics: nDCG at 10, precision at 5, agreement between score bands and recruiter ratings.
- Product metrics: paid placements, 6-month retention, time to first employer response, reported income.
- Pilot with 50 employers and 200 young people. These are pilot parameters, not findings. Compare scoring weights before locking them.

11. Proposed roadmap

Phase	Timing	Deliverables
1 Core	Weeks 1 to 4	Taxonomy, profiles, listings, REST API, keyword plus vector search, basic score
2 Intelligence	Weeks 5 to 8	MCP server, LLM skill extraction, evidence verification, explanations
3 Pilot	Weeks 9 to 12	50 employers, 200 youth, labelled set, weight calibration, fairness report
4 Reach	After pilot	WhatsApp or USSD channel, mobile-money payments for freelance work, more countries

12. Risks and mitigations

Risk	Mitigation
Cold start, no employers	Recruit employers first. Start from real hiring demand, then onboard talent
Biased scoring	Excluded features, subgroup audits, visible breakdowns
Fake profiles or listings	Evidence verification, employer checks, reporting
LLM hallucination	Grounded explanations, structured output, schema validation
Prompt injection through profile text	Treat as data, sanitise, no tool access from profile content
Low connectivity	Light pages, SMS and WhatsApp channels, partner access points
Duplicates existing job boards	Differentiate on verified skills and tracked outcomes